

AI Usage & Disclosure Statement

1. General Disclosure

GAI tool(s) used: Claude (Anthropic)

Model / version: Claude Sonnet 5

Interface: [Claude.ai](https://claude.ai)

Date(s) consulted: September 24, 2026 - September 27, 2026

2. Code Attribution

2.1 Prompt Log

Record of every prompt that produced code or a meaningful revision, in order.

Iteration #	Prompt given to AI (summary)	AI code output:
1	Establish program description restrictions	None
2	Define what a proper IPv4 address should contain. And what the input handler should expect as a user input.	None
3	Clarify decimalValue type, END input case, and return values.	None
4	List of conditions that would result in a invalid input	None
5	Check the AI context of the project requirements.	None
6	Clarify input assumptions	None
7	Share current main.c file and check for errors	None
8	Create and implement extractIPv4	main.c with a new extractIPv4 function
9	Review iteration changes	None

2.2 Full Prompt Text

Iteration 1 — full prompt:

I want to build a C program that reads a line of string from the user and extracts a single valid IPv4 address within it.

Here are some restrictions for the project. Do not use the following:

Any string-to-number conversion function: atoi, atol, atoll, strtol, strtoul, strtod, stoi, stol, stoul, sscanf, scanf with numeric conversions.

Any address-parsing library function: inet_aton, inet_pton, inet_addr, or equivalents.

Any regular-expression facility (std::regex, POSIX regex.h, or similar). The parsing and validation logic must be your own character-by-character code, not a pattern matched by a library.

Don't generate any code until I tell you. Follow along as I add on the program requirements.

Iteration 2 — full prompt:

The input will be a string that the user inputs into the command line. The string may contain letters, digits, symbols, and special characters.

The input handler must be able to focus only on digits, periods, and colons, as the standard format of an IPv4 address could be `192.168.1.30` or `192.168.1.30:500`. To be more specific the address has four octets separated by periods. Each is octet 1–3 digits, value 0–255, and no leading zero unless the value is exactly 0. And optionally a port starting with a colon and an octet containing 1–5 digits, value 0–65535, same leading-zero rule.

The address must match the full IPv4 grammar. If not, print Invalid input: no valid IPv4 address found. Else print `Extracted IPv4 address: <IPv4Address> (decimal value: <decimalValue>, port: <optionalPort>)`. <optionalPort> can be set to -1 if there is no port value.

Still don't generate any code. Wait until I tell you.

Iteration 3 — full prompt:

Also add that the <decimalValue> should be in a 32-bit decimal value. If the user enters END as an input, then print Program terminated and exit. The Input handler should also be called extractIPv4. The prints I mentioned before should be done instead in main, so have the `extractIPv4` just update the values char <IPv4Address>, int <decimalValue> and int <optionalPort> from a struct IPv4 Address. If the input is Invalid return a char `-1` as the <IPv4Address>. And I'll configure the if statements in main and the struct IPv4 address.

Iteration 4 — full prompt:

Here is also a list of what would cause an invalid input/IPv4 address:

- Wrong octet count
- Empty octet
- Out-of-range octet or port
- Leading zero
- Second colon
- Colon not immediately after the fourth octet
- Stray period/colon

Iteration 5 — full prompt:

What clarifications do you need? And what other conditions are you assuming that I haven't specified to you yet?

Iteration 6 — full prompt:

1. It is assumed that only one IPv4 address is entered at a time. So if there is a second address in the same input the program should only fail or return values based on the first address it finds.
2. The start of the address should be a string of digits between 0-9 followed by a period, so ignore any letters, -, /, or whitespace at the start of the input. So the address only contains from start to finish these characters [0-9.:]. Then each potential octet should be checked for the number of digits, value, and no leading zero unless the value is exactly 0.
3. There should only exist one colon.
4. Once you identified the end of the last octet or port, any other characters and whitespace after that should be ignored. Unless a stray period/colon is directly adjacent to the address. Then that should also result in a invalid address

Iteration 7 — full prompt:

Here is the code I made for the main loop and IPv4Address struct. I left the extractIPv4 function that takes the char input pointer as a parameter. Let me know if you see any errors. Again don't generate any code.

(Attached main.c with the prompt)

Iteration 8 — full prompt:

Now you can generate code for the extractIPv4 function. Only edit the function and add the proper function call in main.

Iteration 9 — full prompt:

I ran some test cases and found some errors with the output. 'isdigit' created compiling issues, because it was not declared correctly. So I've added the '<ctype.h>' to the library list. The output value for decimal value is negative, so I changed the struct decimalValue type to also be unsigned int. Here is how the updated code looks like. Don't make changes to it.

(Attached main.c with the prompt)

2.3 AI-generated versus student-written code

List what sections of the code were done entirely by AI or the student.

AI-generated:

- ExtractIPv4 function
 - Find the first octet of the address
 - Check all four octets for validity
 - Check for OPTIONALport
 - Build the final address string and its decimal value

Student-written:

- IPV4 Address Struct
- Main loop
 - User input
 - Program print output
- Comments to code made by student and AI-generated

2.4 AI code modified by student

List what sections of the code listed in section 2.3 were done AI generation and then student modified.

Iteration 8 — Implicit declaration

The main.c file would not compile for testing. Looking into the code the AI used the function `isdigit()` without including the C library `<ctype.h>`. So the AI caused an implicit declaration of the function.

This was fixed by adding the line `#include <ctype.h>` with the other libraries at the very top of the file.

Iteration 8 — int decimalValue positive value overflow

On testing the user input the decimal value would return as a negative value.

Example:

Enter a string (or 'END' to quit): 192.127.1.30

Extracted IPv4 address: 192.127.1.30 (decimal value: -1065418466, port: -1)

The AI calculates the decimalValue as an `'unsigned int'` variable, which it later converts to a regular `'int'` for the struct type that decimalValue has. So I changed the struct decimalValue type to also be `'unsigned int'` and the printf format identifier to `'%u'`.

Iteration 9 — Input 192.168.1.1. passes when it should not

AI didn't implement the check for the adjacent stray period or colon next to the last octet (including the port octet if included). So the test case `'192.168.1.1.'` results in the output:

Extracted IPv4 address: 192.168.1.1 (decimal value: 3232235777, port: none)

So I added another check for the adjacent dot or colon with an if statement right after the port section of code.

3. Verification statement

- I confirm that you understand every line of the submitted code, whether that code was done solely by myself, AI, or in collaboration with AI.
- I confirm that code has been tested and passes defined test cases made for the purpose to ensure the programs works as intended
- I acknowledge any known bugs, limitations, or unexpected behavior not resolved in the final iteration of the program.